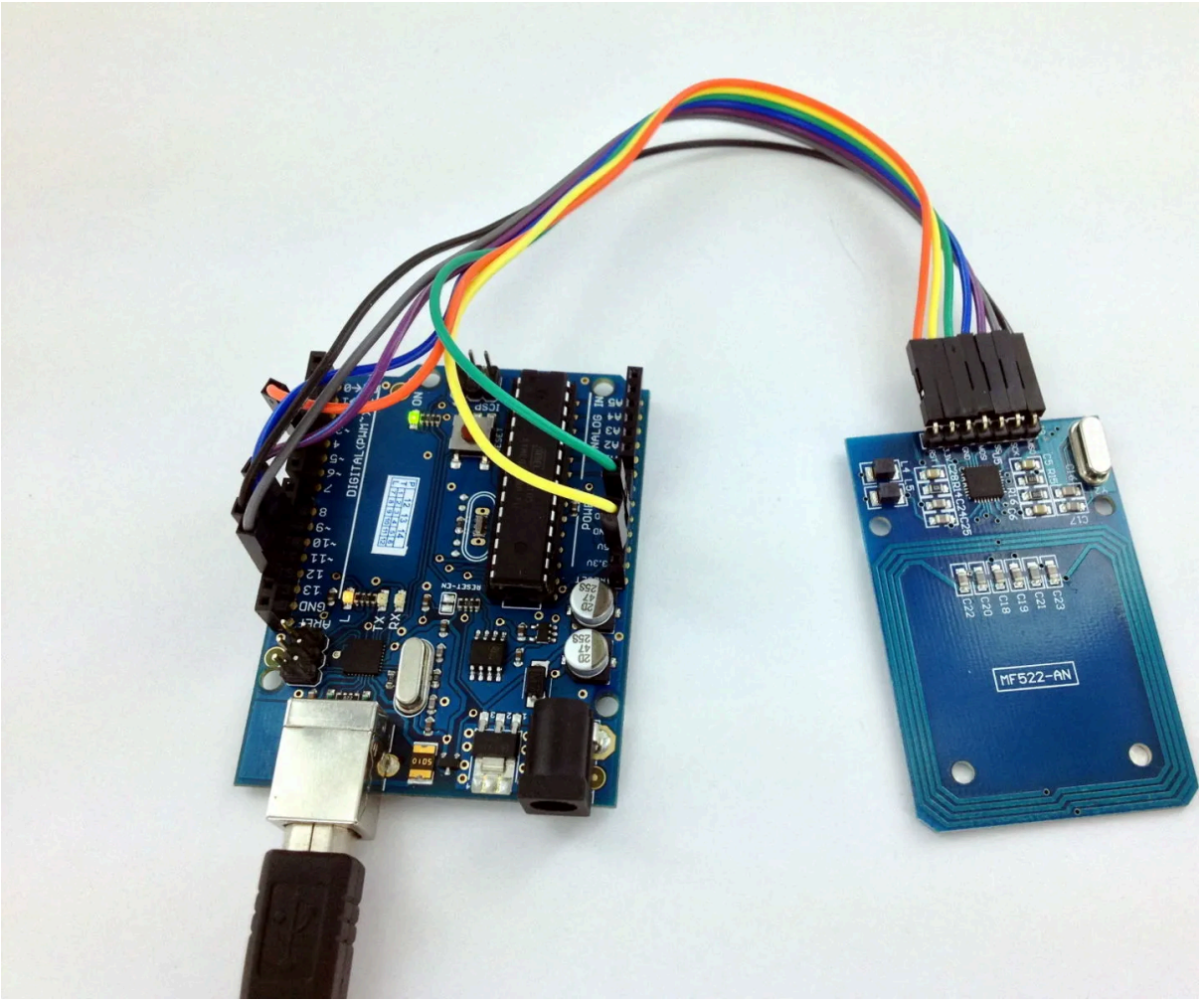
Updated Component List & Design Modification



Design Change Overview

In the initial design, the **spin-the-wheel mechanism** used **9 reed switches** along with a **magnet** attached to the rotating wheel to detect positions.

However, this setup required extensive wiring and created circuit complexity on the breadboard.

To improve reliability, wiring efficiency, and system clarity, the reed switch mechanism was replaced with an **RFID-based selection system**.

⚙️ Previous Mechanism (Removed)

Components Removed:

- 9 × Reed Switches
- 1 × Permanent Magnet (mounted on wheel)

Working Principle:

- The rotating wheel carried a magnet.
- Each reed switch detected the magnet when it passed nearby.
- Three reed switches were assigned per color.
- Each group of three switches controlled one LED color.
- All switches worked independently but were connected in parallel to a single LED output.

Problems Faced:

- Required **18 breadboard connection points** for reed switches alone.
- Heavy wire congestion.
- Difficult troubleshooting.
- Mechanical alignment issues between magnet and switches.
- High chance of false triggering due to vibration or positioning errors.
- Overall circuit became messy and less scalable.

✅ Updated Mechanism (Implemented)

New Components Added:

- 1 × RFID Reader Module (RC522)
- 3 × RFID Room Key Cards
- Jumper wires (minimal wiring required)

Working Principle:

- Each RFID card has a **unique UID**.
- Three RFID cards are mapped digitally:
    - Card UID 1 → Color A
    - Card UID 2 → Color B
    - Card UID 3 → Color C
- When a card is scanned, the ESP32 reads the UID and activates the corresponding LED/candy dispenser.

Reed Switch System	RFID System
Mechanical sensing	Digital identification

18+ breadboard points	Minimal GPIO usage
Complex wiring	Clean architecture
Alignment dependent	Contactless detection
Hard to expand	Easily scalable
Prone to misfires	Reliable UID detection

🚀 Advantages of RFID-Based Design

- Cleaner hardware layout.
- Significant reduction in wiring complexity.
- Faster debugging and maintenance.
- Improved reliability during demonstrations.
- Easier mapping of game logic in software.
- Professional competition-level architecture.